



```
In [61]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import scipy.stats as stats
```

```
In [62]: from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score

from sklearn.linear_model import LinearRegression

from sklearn.metrics import r2_score

from sklearn.preprocessing import PowerTransformer
```

```
In [63]: df = pd.read_csv("concrete_data.csv")
```

```
In [64]: df.head()
```

```
Out[64]:
```

	Cement	Blast Furnace Slag	Fly Ash	Water	Superplasticizer	Coarse Aggregate	Fine Aggregate	Age
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0	270
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0	365
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5	360

```
In [65]: df.shape
```

```
Out[65]: (1030, 9)
```

```
In [66]: df.isnull().sum()
```

```
Out[66]: Cement          0
Blast Furnace Slag      0
Fly Ash                 0
Water                   0
Superplasticizer        0
Coarse Aggregate        0
Fine Aggregate          0
Age                     0
Strength                0
dtype: int64
```

```
In [67]: # there is no any null value in the dataset
```

```
In [68]: df.describe()
```

```
Out[68]:
```

	Cement	Blast Furnace Slag	Fly Ash	Water	Superplasticizer	Age
count	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000
mean	281.167864	73.895825	54.188350	181.567282	6.204660	97.901631
std	104.506364	86.279342	63.997004	21.354219	5.973841	74.595886
min	102.000000	0.000000	0.000000	121.800000	0.000000	80.000000
25%	192.375000	0.000000	0.000000	164.900000	0.000000	93.000000
50%	272.900000	22.000000	0.000000	185.000000	6.400000	96.000000
75%	350.000000	142.950000	118.300000	192.000000	10.200000	102.000000
max	540.000000	359.400000	200.100000	247.000000	32.200000	114.000000

there is some columns which has min value is 0 but no nay
negative value is there (kyu ki agar negative value rehaega tu box-cox
nahi lage ga)

```
In [70]: x = df.drop(columns=['Strength'])  
y = df.iloc[:, -1]
```

```
In [71]: x_train,x_test,y_train,y_test = train_test_split(x,y,  
                                                         test_size=0.2,  
                                                         random_state=42)
```

```
In [72]: # applying Linera Regression without any transformation  
lr = LinearRegression()  
  
lr.fit(x_train,y_train)  
  
y_pred = lr.predict(x_test)  
  
r2_score(y_test,y_pred)
```

```
Out[72]: 0.6275531792314848
```

```
In [73]: # Cross checking with cross val score  
lr = LinearRegression()  
np.mean(cross_val_score(lr,x,y,scoring='r2'))
```

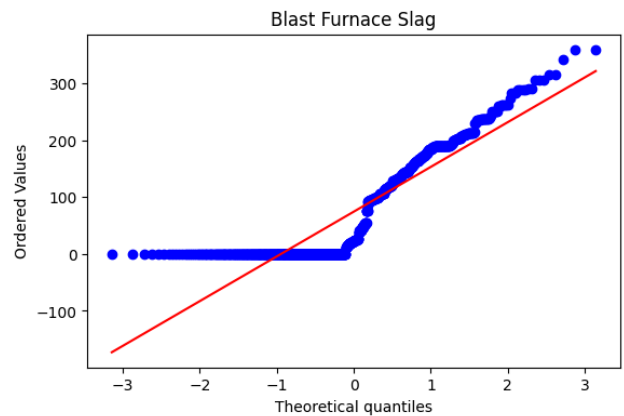
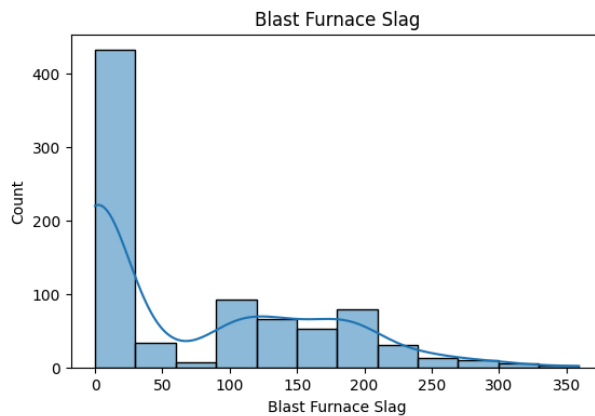
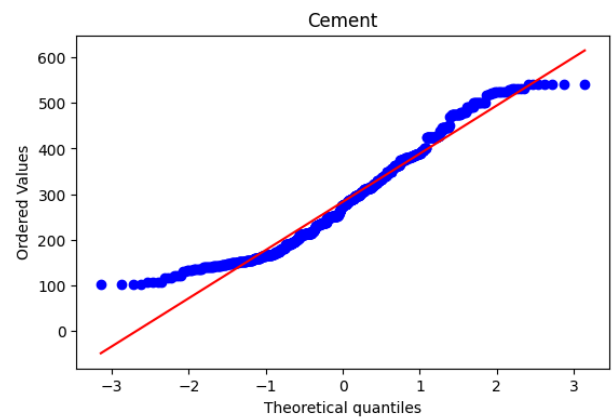
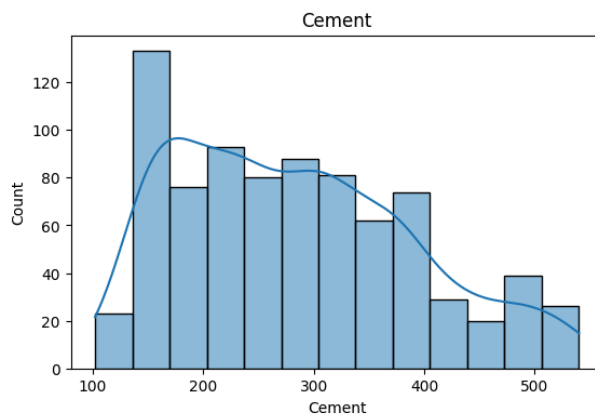
```
Out[73]: 0.4609940491662864
```

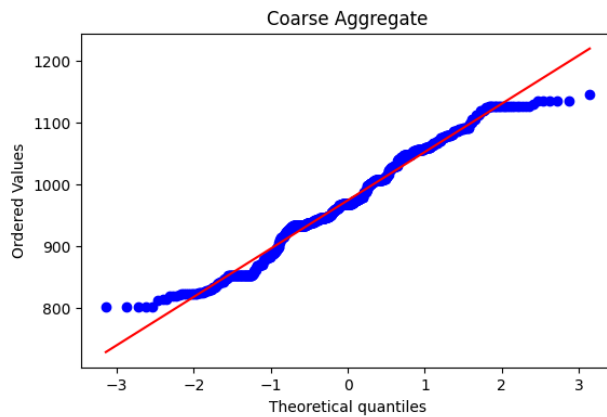
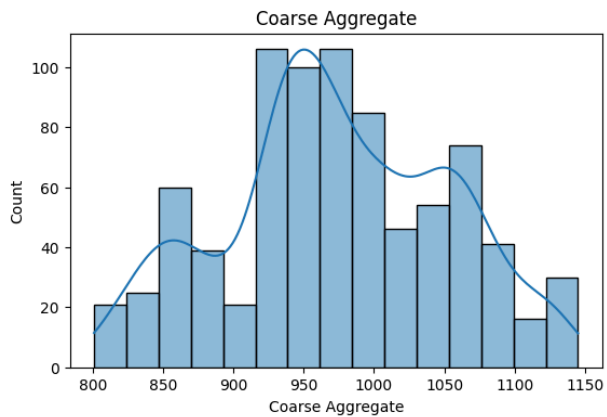
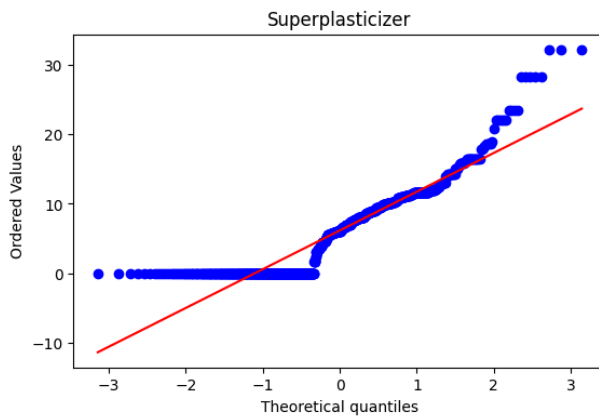
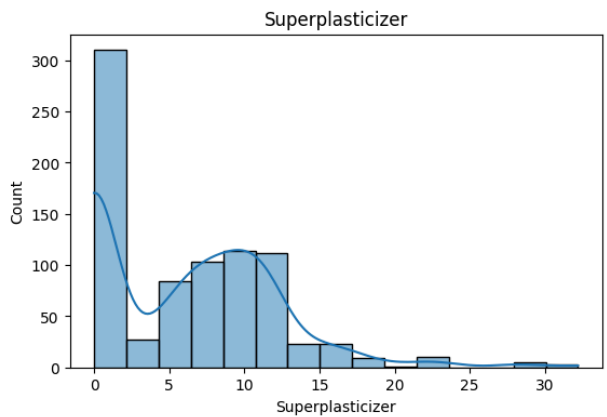
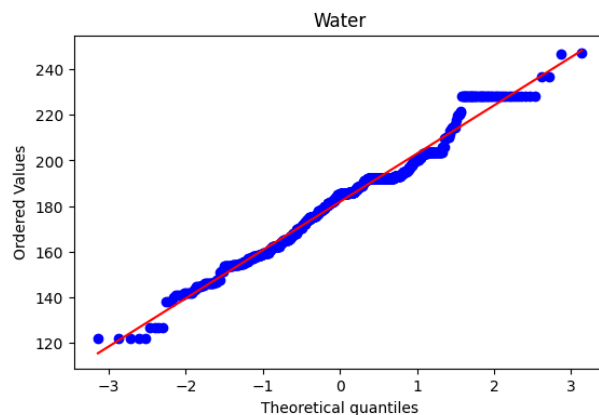
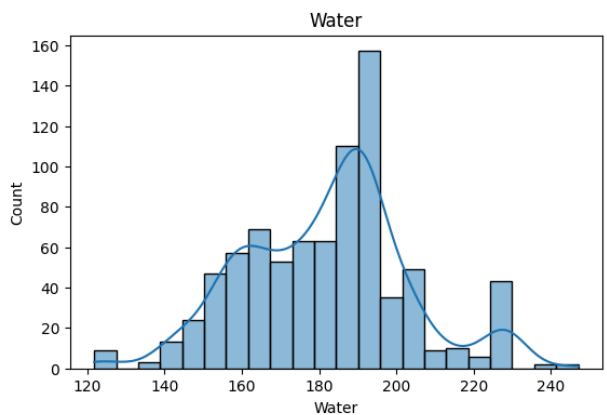
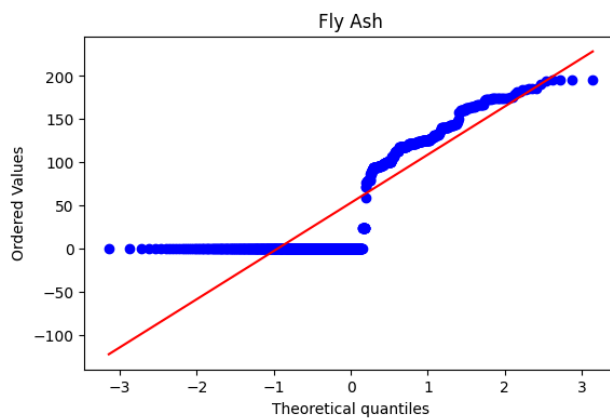
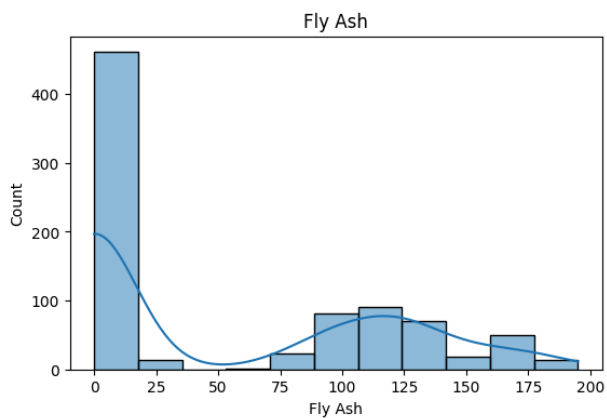
```
In [ ]:
```

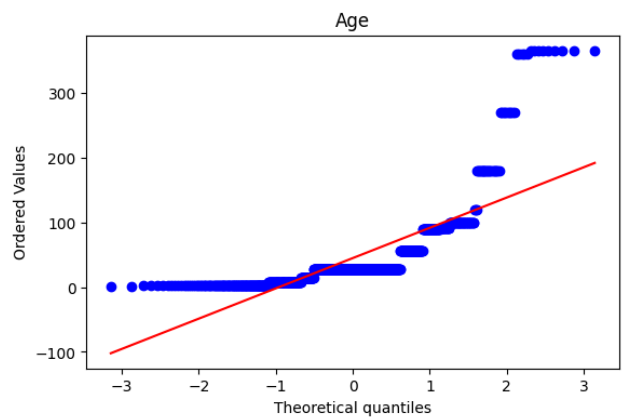
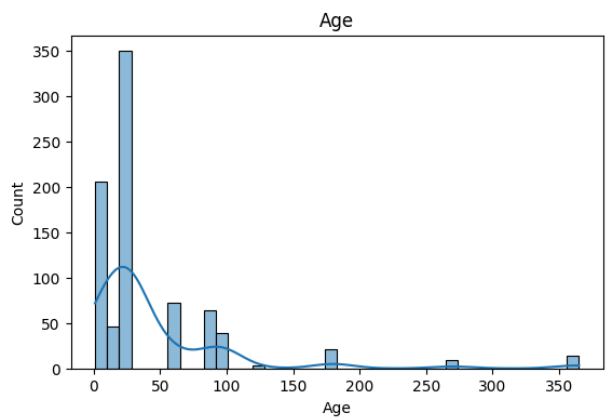
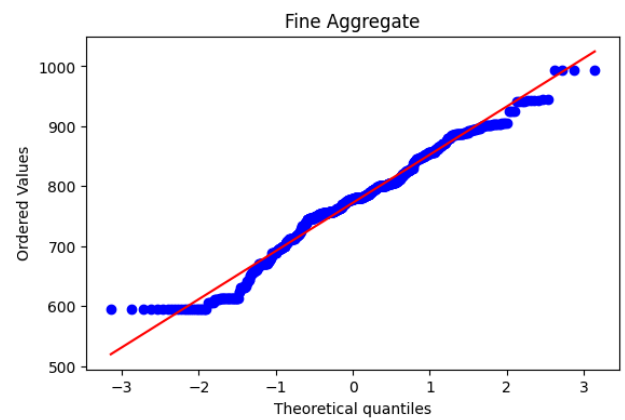
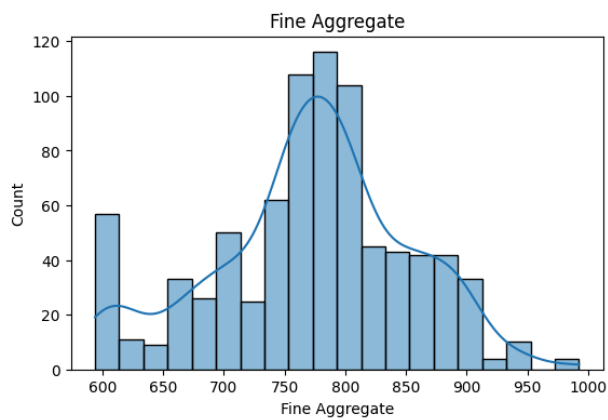
```
In [ ]:
```

```
In [74]: # now plotting the graph without any transformation
```

```
In [75]: for col in x_train.columns :  
    plt.figure(figsize=(14,4))  
  
    # Plot 1: Distribution  
    plt.subplot(1,2,1)  
    sns.histplot(x_train[col], kde=True)  
    plt.title(col)  
  
    # Plot 2: QQ Plot  
    plt.subplot(1,2,2)  
    stats.probplot(x_train[col], dist="norm", plot=plt)  
    plt.title(col)  
  
    plt.show()
```







Applying Box-Cox transformation

```
In [77]: pt = PowerTransformer(method='box-cox')
```

```
x_train_transformed = pt.fit_transform(x_train+0.00001)
x_test_transformed = pt.fit_transform(x_test+0.00001)
```

```
In [78]: pd.DataFrame({'cols':x_train.columns,'box_cox_lambdas':pt.lambdas_})
```

```
Out[78]:
```

	cols	box_cox_lambdas
0	Cement	0.215602
1	Blast Furnace Slag	0.025996
2	Fly Ash	-0.006544
3	Water	0.959062
4	Superplasticizer	0.103365
5	Coarse Aggregate	1.192492
6	Fine Aggregate	1.973781
7	Age	-0.014692

```
In [79]: # now applying linear regression on transformed data
```

```

lr = LinearRegression()
lr.fit(x_train_transformed,y_train)

y_pred2 = lr.predict(x_test_transformed)

r2_score(y_test,y_pred2)

```

Out[79]: 0.8060839821745337

```

In [80]: # Using cross val score

pt = PowerTransformer(method='box-cox')
x_transformed = pt.fit_transform(x+0.0000001)

lr = LinearRegression()
np.mean(cross_val_score(lr,x_transformed,y,score='r2'))

```

Out[80]: 0.6658537941434357

```

In [81]: # converting numpy array to dataframe
x_train_transformed = pd.DataFrame(x_train_transformed,columns=x_train.columns)

```

```

In [82]: # Before and after box-cox Transformation comparision

```

```

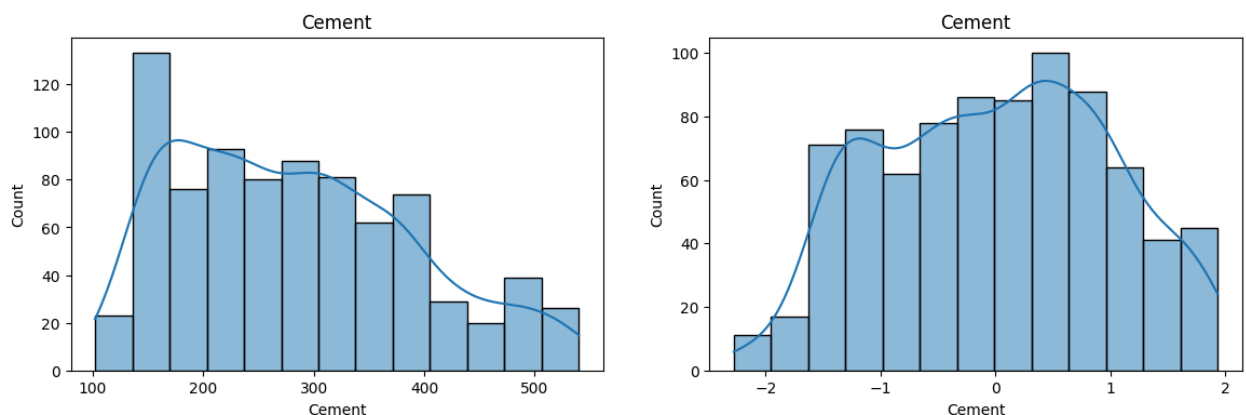
for col in x_train_transformed.columns :
    plt.figure(figsize=(14,4))

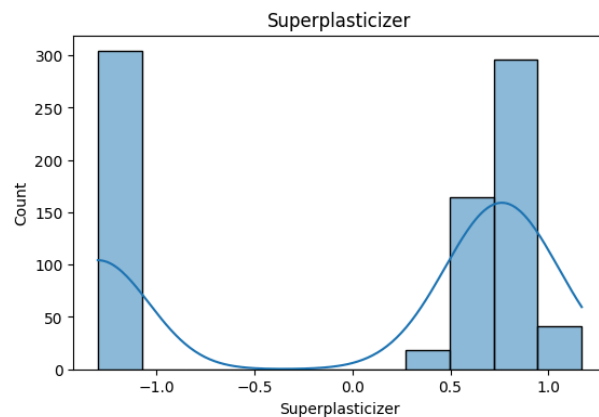
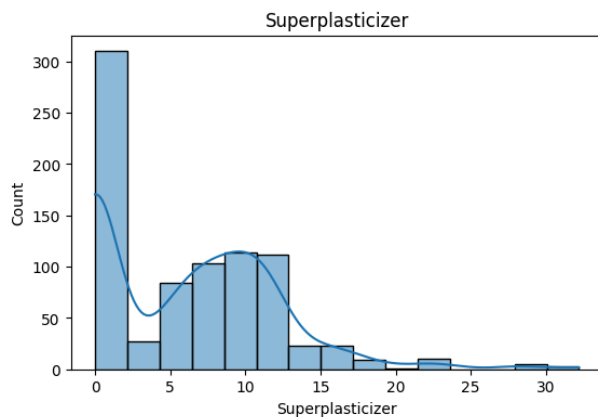
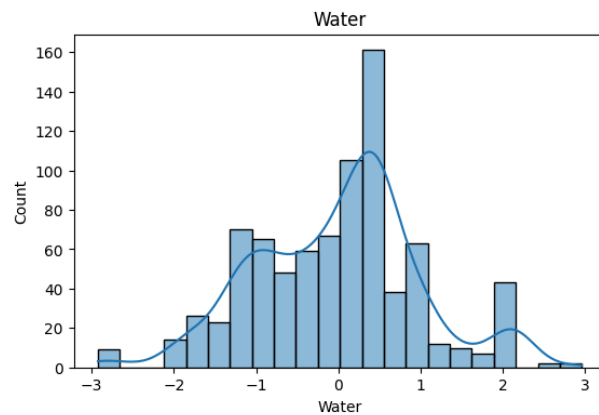
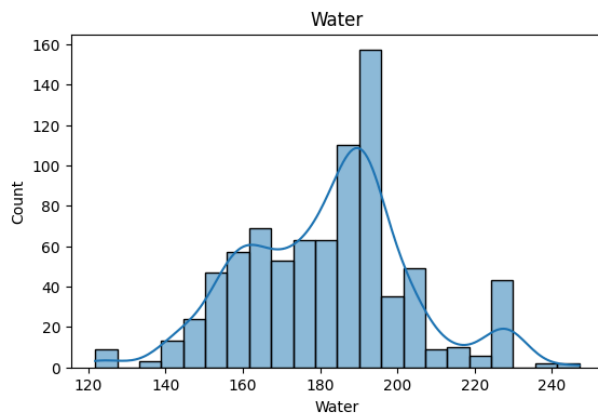
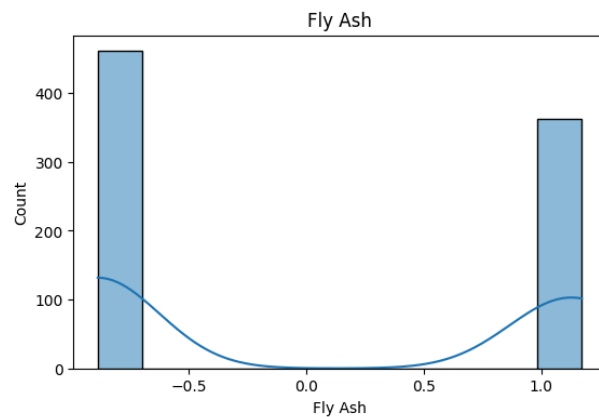
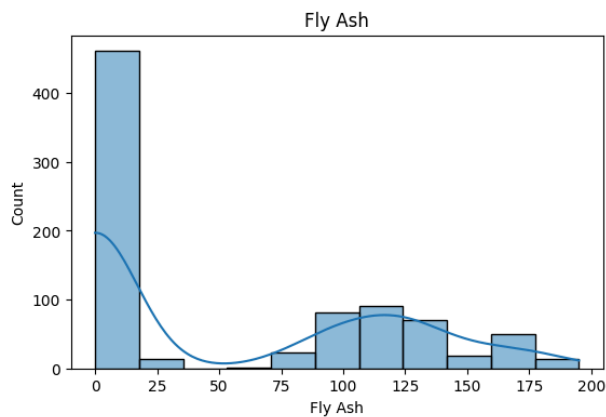
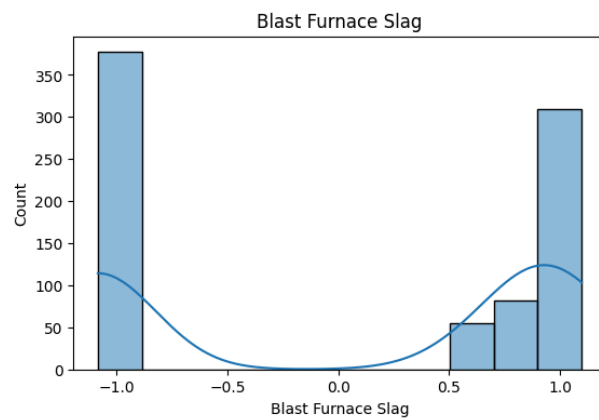
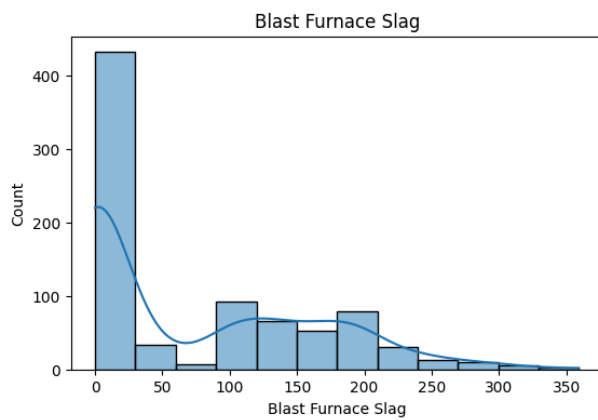
    # Plot 1: Distribution
    plt.subplot(1,2,1)
    sns.histplot(x_train[col], kde=True)
    plt.title(col)

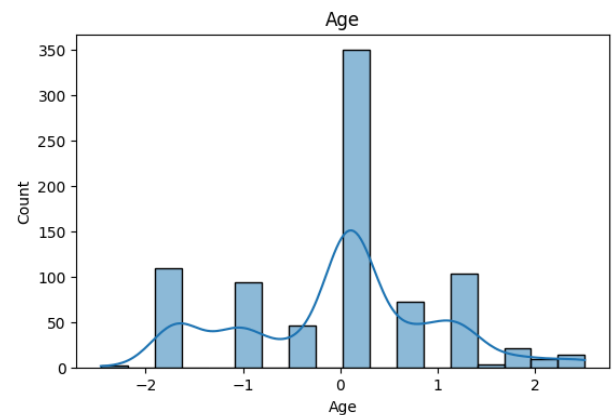
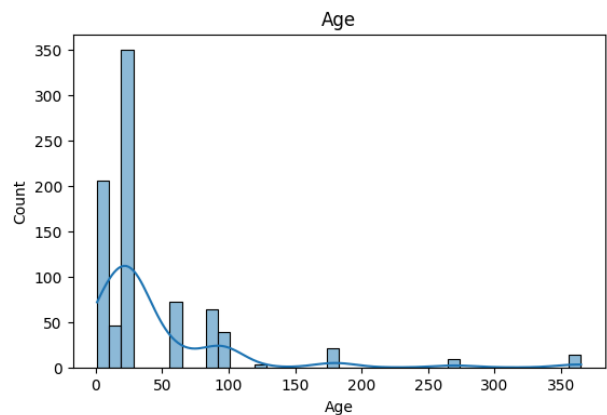
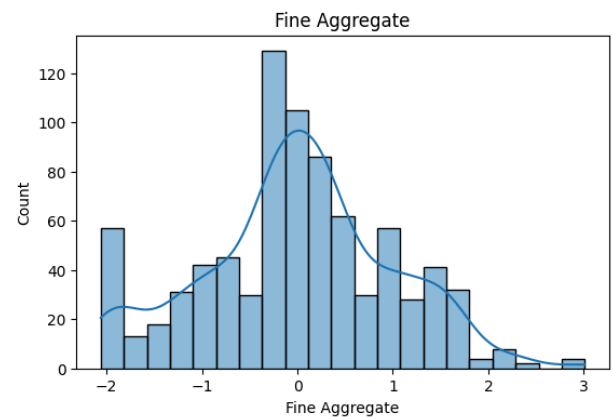
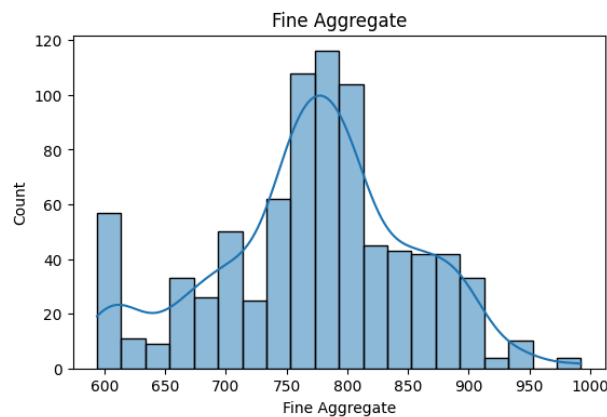
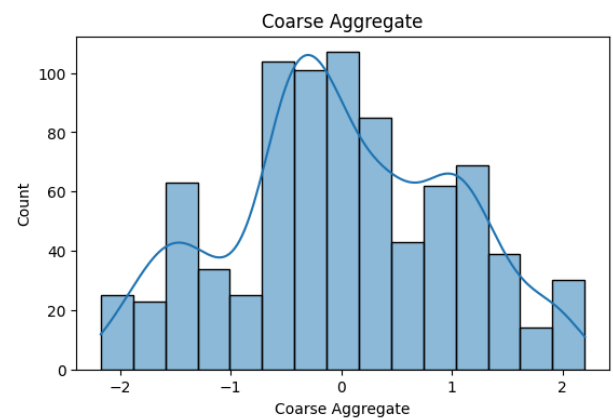
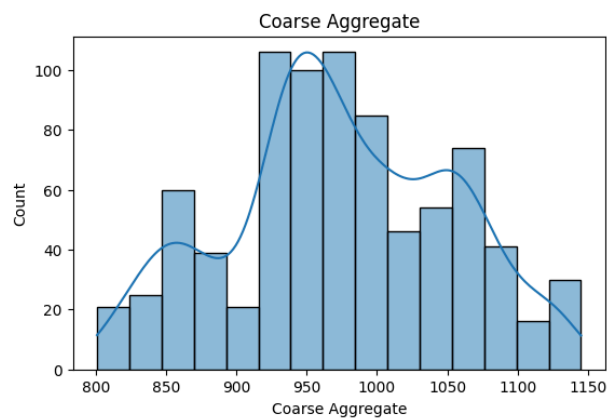
    # Plot 2: distribution
    plt.subplot(1,2,2)
    sns.histplot(x_train_transformed[col], kde = True)
    plt.title(col)

    plt.show()

```







Applying Yeo jhonson transform

```
In [84]: pt1 = PowerTransformer() # by default transformer in this is yeo jhonson

x_train_transformed2 = pt1.fit_transform(x_train)
x_test_transformed2 = pt1.fit_transform(x_test)

lr = LinearRegression()
lr.fit(x_train_transformed2,y_train)

y_pred3 = lr.predict(x_test_transformed2)

print(r2_score(y_test ,y_pred3))
```

```
pd.DataFrame({'cols':x_train.columns,'yeo_jhonson_lambdas':pt1.lambdas_})
```

0.8153845514645103

Out[84]:

	cols	yeo_jhonson_lambdas
0	Cement	0.213025
1	Blast Furnace Slag	0.020476
2	Fly Ash	-0.038218
3	Water	0.958936
4	Superplasticizer	0.304951
5	Coarse Aggregate	1.192751
6	Fine Aggregate	1.975085
7	Age	-0.062443

```
In [85]: # applying cross val score

pt = PowerTransformer()
x_transformed2 = pt.fit_transform(x)

lr = LinearRegression()
np.mean(cross_val_score(lr,x_transformed2,y,scoring='r2'))
```

Out[85]: 0.6834625126992433

```
In [86]: # Again converting numpy array to dataframe
x_train_transformed2 = pd.DataFrame(x_train_transformed2,columns=x_train.columns)
```

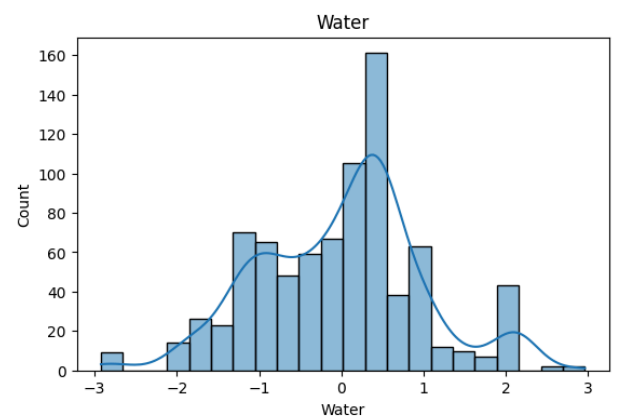
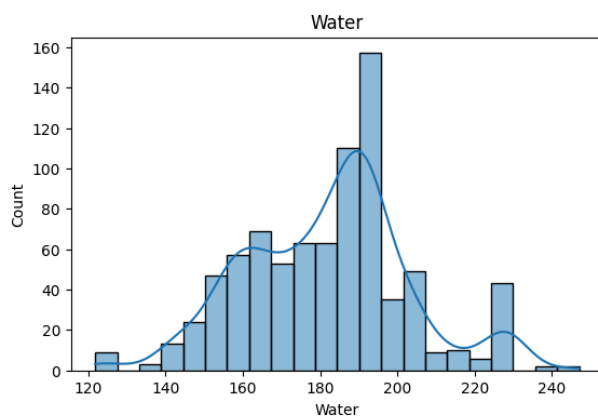
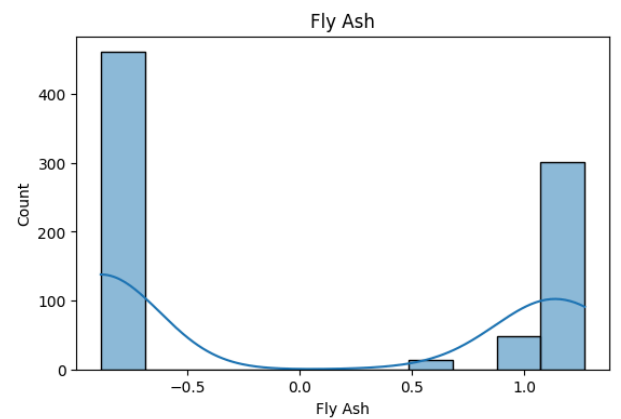
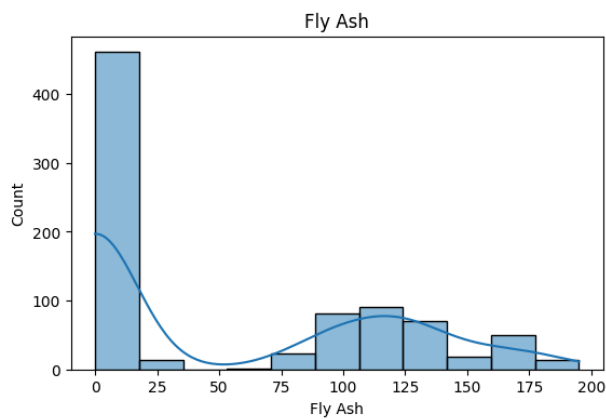
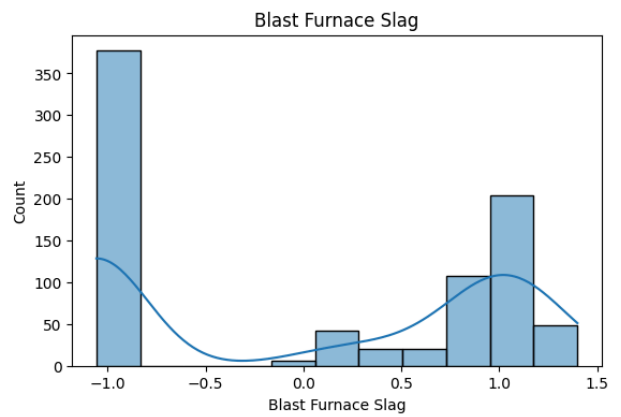
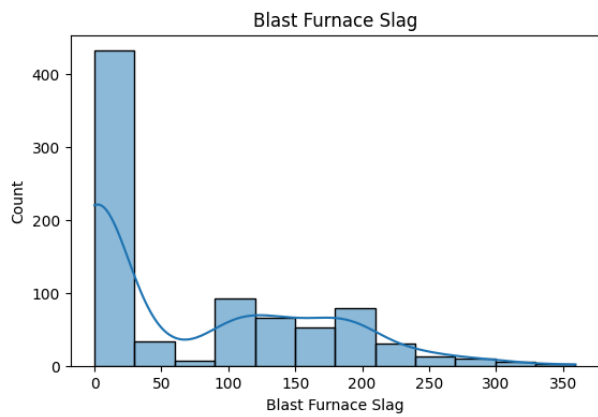
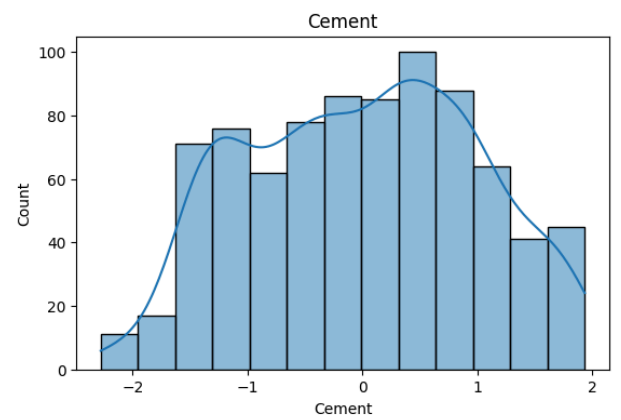
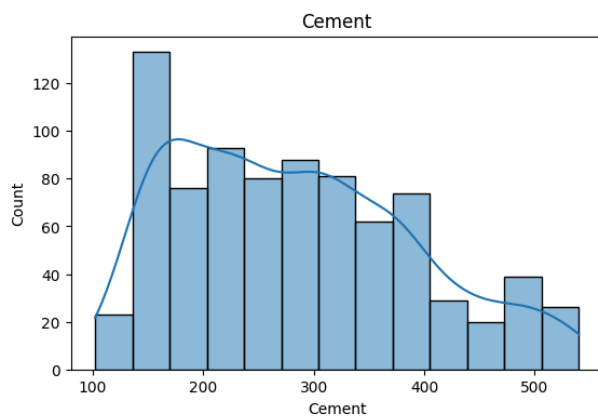
```
In [87]: # Before and after yeo-jhonson Transformation comparision

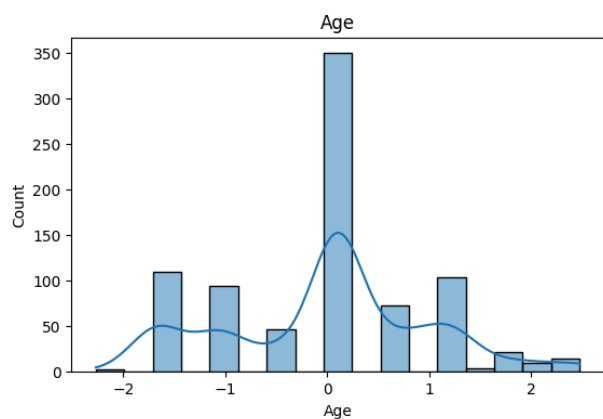
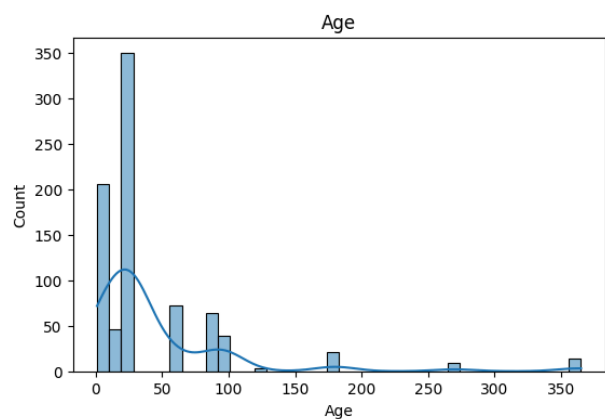
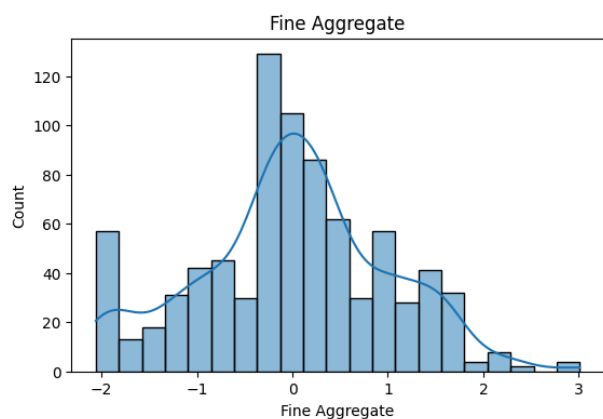
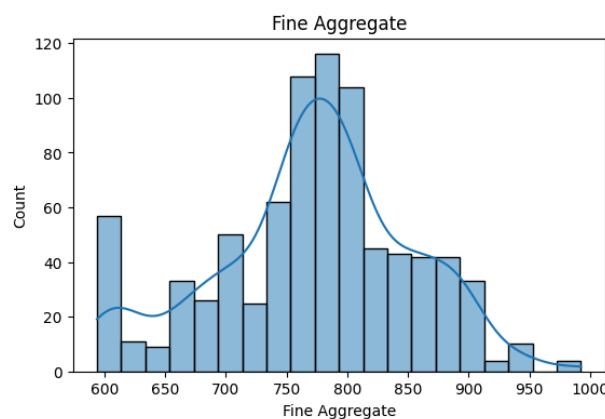
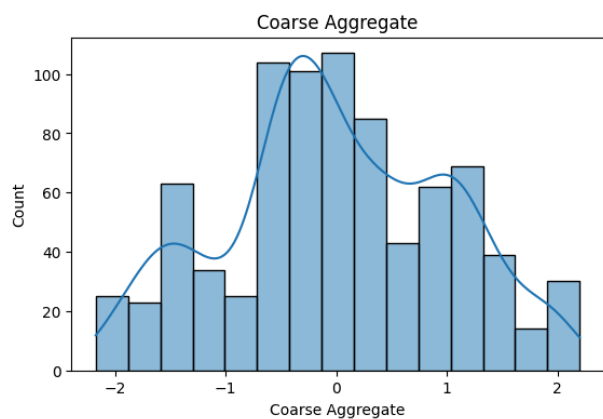
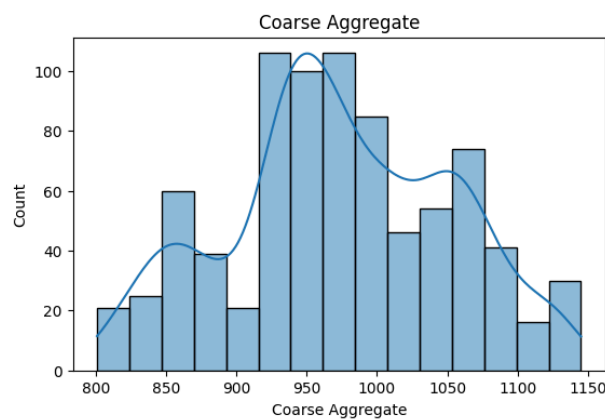
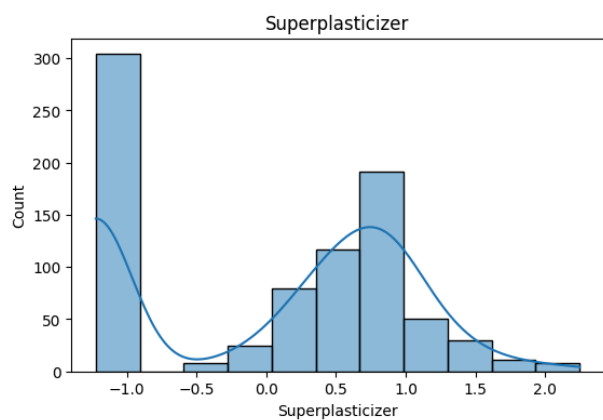
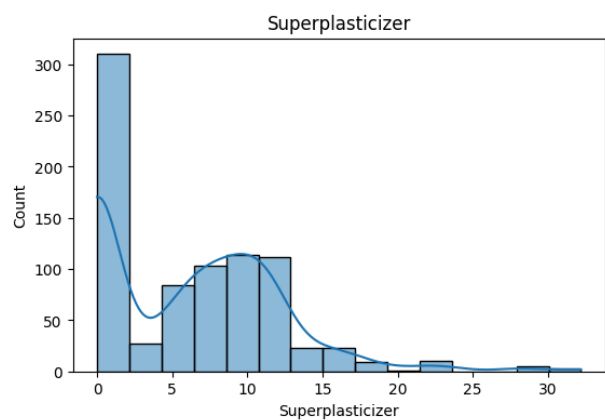
for col in x_train_transformed2.columns :
    plt.figure(figsize=(14,4))

    # Plot 1: Distribution
    plt.subplot(1,2,1)
    sns.histplot(x_train[col], kde=True)
    plt.title(col)

    # Plot 2: distribution
    plt.subplot(1,2,2)
    sns.histplot(x_train_transformed2[col], kde = True)
    plt.title(col)

plt.show()
```





```
In [88]: # Side by side Lambdas
pd.DataFrame({'cols':x_train.columns,'box_cox_lambdas':pt.lambdas_,
```

```
'Yeo_Johnson_lambdas':pt1.lambdas_})
```

Out[88]:

	cols	box_cox_lambdas	Yeo_Johnson_lambdas
0	Cement	0.169544	0.213025
1	Blast Furnace Slag	0.016633	0.020476
2	Fly Ash	-0.136480	-0.038218
3	Water	0.808438	0.958936
4	Superplasticizer	0.264160	0.304951
5	Coarse Aggregate	1.129395	1.192751
6	Fine Aggregate	1.830764	1.975085
7	Age	0.001771	-0.062443

In []: